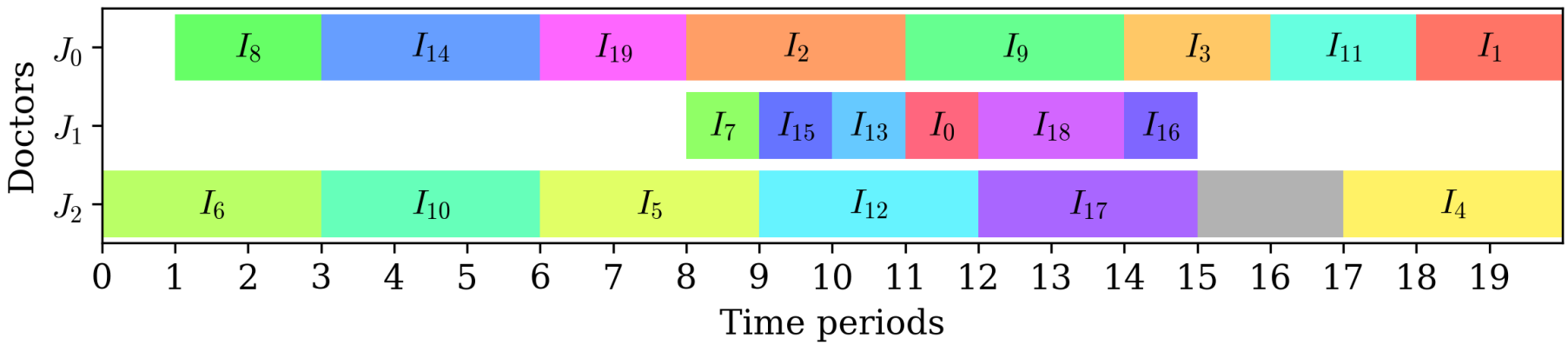
Doctor-patient matching & scheduling optimisation problem

Peleg Kark-Levin Tyler Pearn Hamish McKay

The University of Queensland

Semester 2, 2025



Project overview

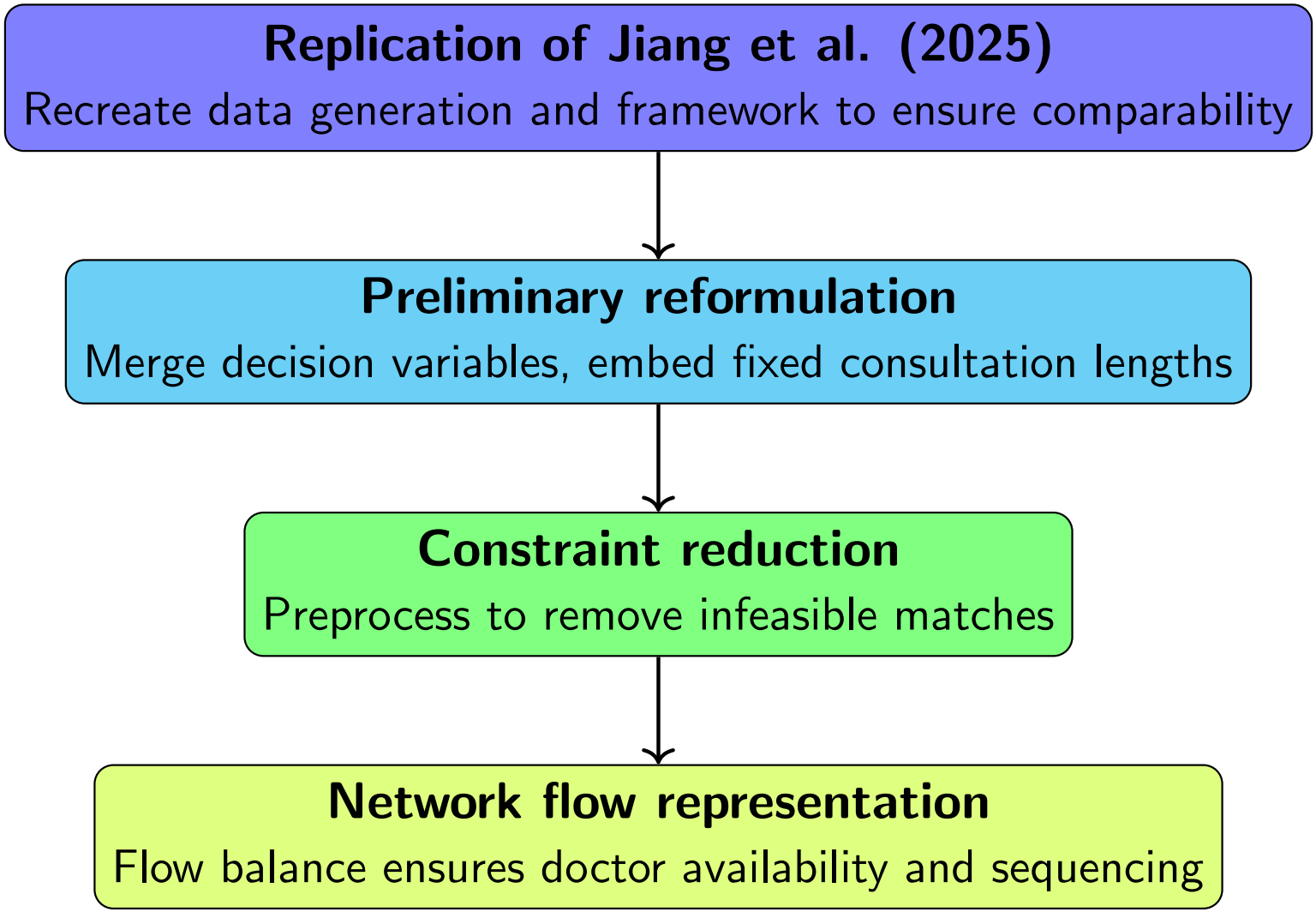
Goal: Reproduce, improve, and extend the model presented by Jiang et al. (2025)[1] on doctor-patient matching and scheduling for online consultations.

Problem summary:

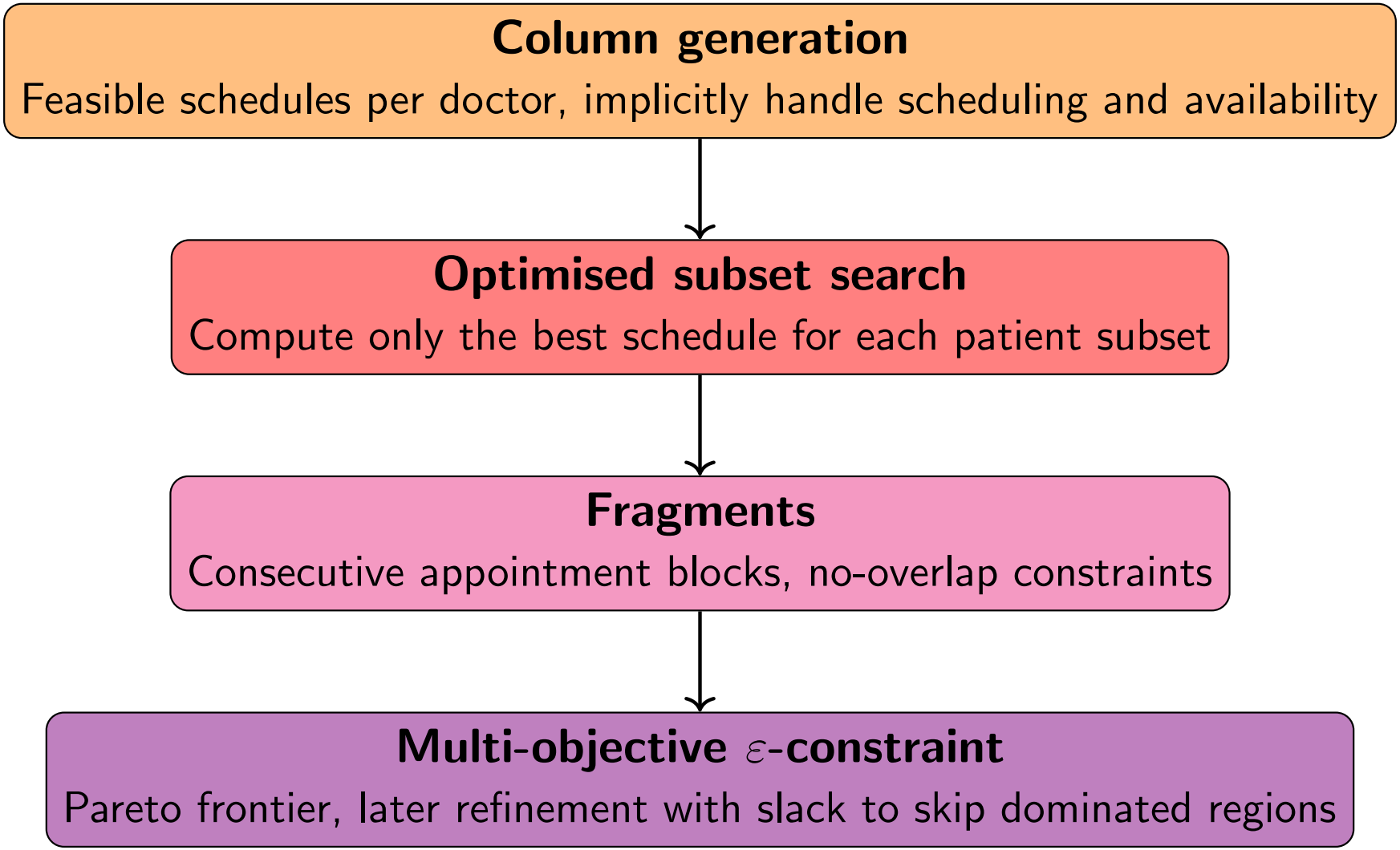
- Multi-objective optimisation: maximise patient satisfaction, doctor satisfaction, and number of appointments.
- Complex constraints: overlapping availability, consultation duration by disease/expertise, and individual preferences.
- High heterogeneity among patients and doctors $\rightarrow$ limited symmetry exploitation, tightly coupled problem.

[1] **Reference:** Jiang et al. (2025). *Logic-based Benders decomposition for doctor-patient matching and scheduling considering chronic patients' online consultation time preference*. Computers & Operations Research 183: 107207. https://doi.org/10.1016/j.cor.2025.107207

Modelling journey



Modelling journey (continued)



Model outline

We model doctor-patient matching as a scheduling problem, balancing patient and doctor preferences across multiple time periods.

Sets:

I	Patients	I_k	Patients with disease k
J	Doctors	J_k	Doctors qualified for k
K	Disease types	$\text{treat}_{j,k}$	Treatment time for doctor j and disease k
T	Time periods		
S	Candidate schedules	$T_{i,j}^{\text{compatible}}$	Feasible start times for patient i and doctor j

Objectives:

- Maximise patient satisfaction, number of appointments, and doctor satisfaction

Illustrative doctor–patient schedule

Overview:

- Each doctor is assigned a sequence of patients over time.
- Patient appointment times are shown in colour blocks.

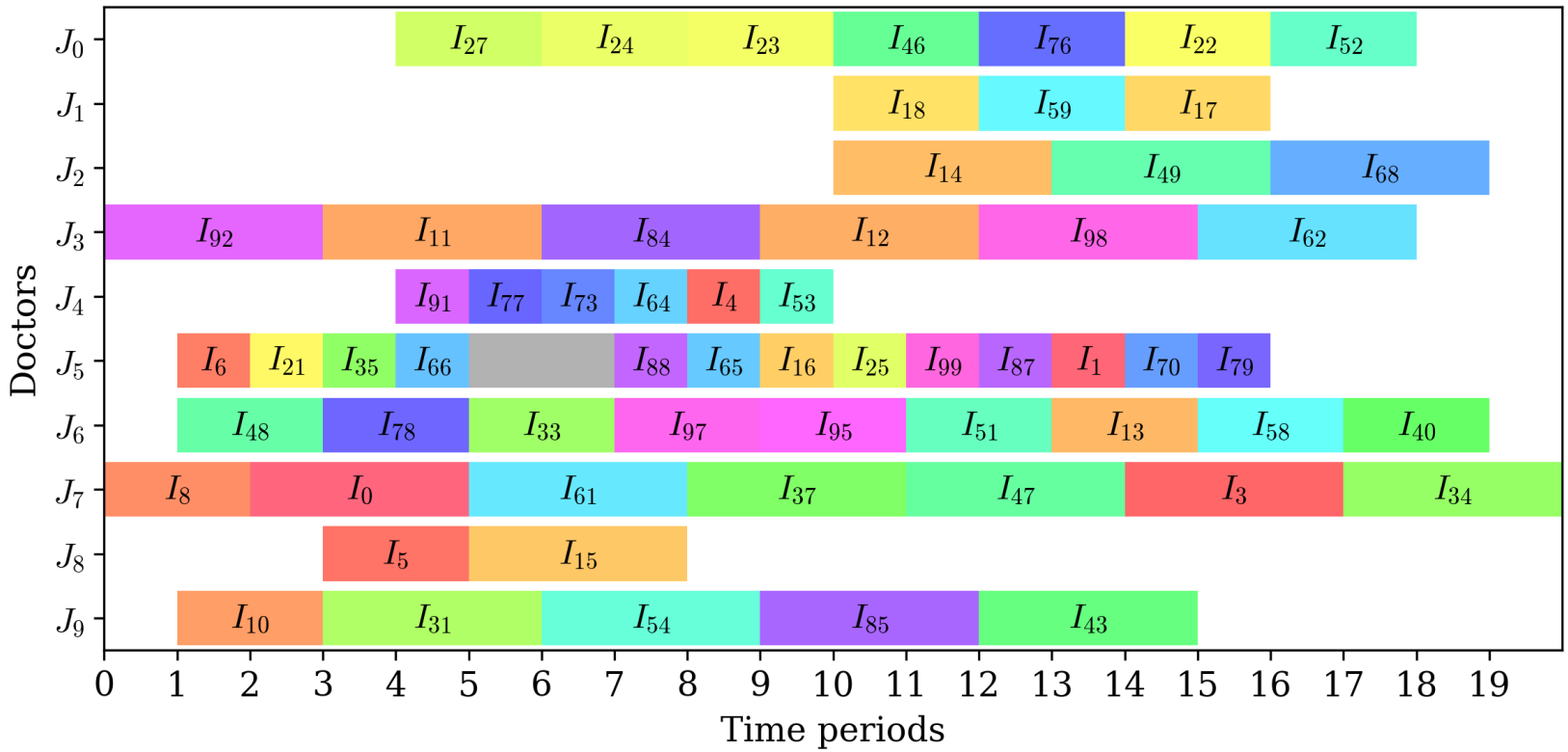


Figure: Example optimal schedule for 10 doctors and 100 patients

Baseline model

Feasibility formulation

Variables:

- Appointment start times:

$$y_{i,j,t} \in \{0,1\} \qquad \forall i \in I,\ j \in J,\ t \in T$$

Constraints:

- Doctors not overbooked
- Patients assigned at most once
- Appointments in feasible time windows
- Only qualified doctor-disease assignments

Preprocessed model

Compatible times formulation

Variables:

- Appointment start times:

$$y_{i,j,t} \in \{0,1\} \qquad \forall k \in K,\ i \in I_k,\ j \in J_k,\ t \in T_{i,j}^{\text{compatible}}$$

Constraints:

- Doctors not overbooked (sum over compatible times only)
- Patients assigned at most once (over compatible doctors and times)

Network flow model

Doctor available formulation

Variables:

- Appointment start times:

$$y_{i,j,t} \in \{0,1\} \qquad \forall k \in K,\ i \in I_k,\ j \in J_k,\ t \in T_{i,j}^{\text{compatible}}$$

- Doctor availability:

$$z_{j,t} \in \{0,1\} \qquad \forall j \in J,\ t \in \text{available}_j^{\text{doctor}}$$

Constraints:

- Patients assigned at most once
- Doctor availability flows over time:
  - $\rightarrow$ $z_{j,t}$ updates based on previous availability and scheduled appointments
  - $\rightarrow$ Initial and final availability constraints ensure feasibility

Network flow model

Doctor available formulation

Doctor availability flows over time:

$$z_{j,t} = z_{j,t-1} + \overbrace{\sum_{k \in D_j} \sum_{\substack{i \in I_k: \\ t^{\text{start}}_{j,k} \in T^{\text{compatible}}_{i,j}}} y_{i,j,t^{\text{start}}_{j,k}}}^{\text{inflows}} - \overbrace{\sum_{k \in D_j} \sum_{\substack{i \in I_k: \\ t \in T^{\text{compatible}}_{i,j}}} y_{i,j,t}}^{\text{outflows}}$$

$$\forall j \in J,\ t \in \text{available}^{\text{doctor}}_j, t^{\text{start}}_{j,k} = t - \text{treat}_{j,k}$$

Model building comparison

	Variables	Constraints	Nonzeros	Setup Time (s)
Feasibility model	20000	40300	148723	0.296
Compatible times model	5332	300	25830	0.058
Doctor available model	5474	252	16281	0.03

Figure: Gurobi model building before presolve(averaged over 1000 iterations)

Key points:

- The feasibility model has a longer set up time and is very sparse
- All three models give the exact same objective values
- Presolve removes the exact same number of rows and columns for each model

Performance profiles

Overview:

- The feasibility model has the steepest slope, with the least variability in solving times across different seeds
- The doctor available model performs best overall, with most instances solved to optimality in the shortest amount of time

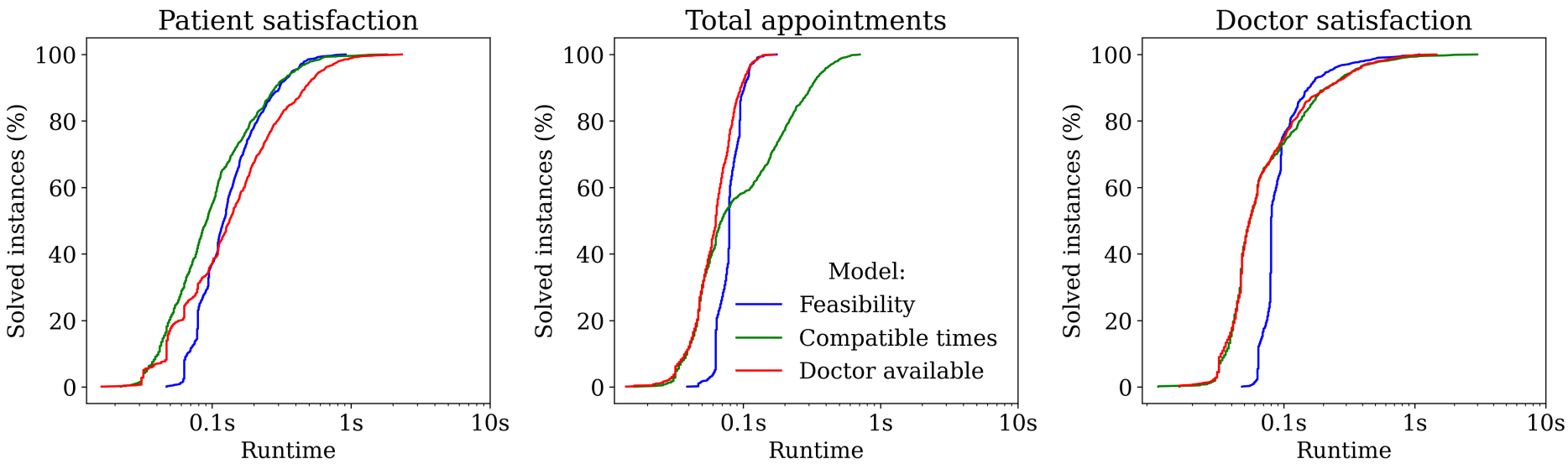


Figure: Performance profiles for compact IP models
(1000 instances with 100 patients, 10 doctors, 4 diseases, and 20 time periods)

Huge model

We developed a huge integer program that selects one feasible schedule per doctor, each representing a full sequence of patient appointments

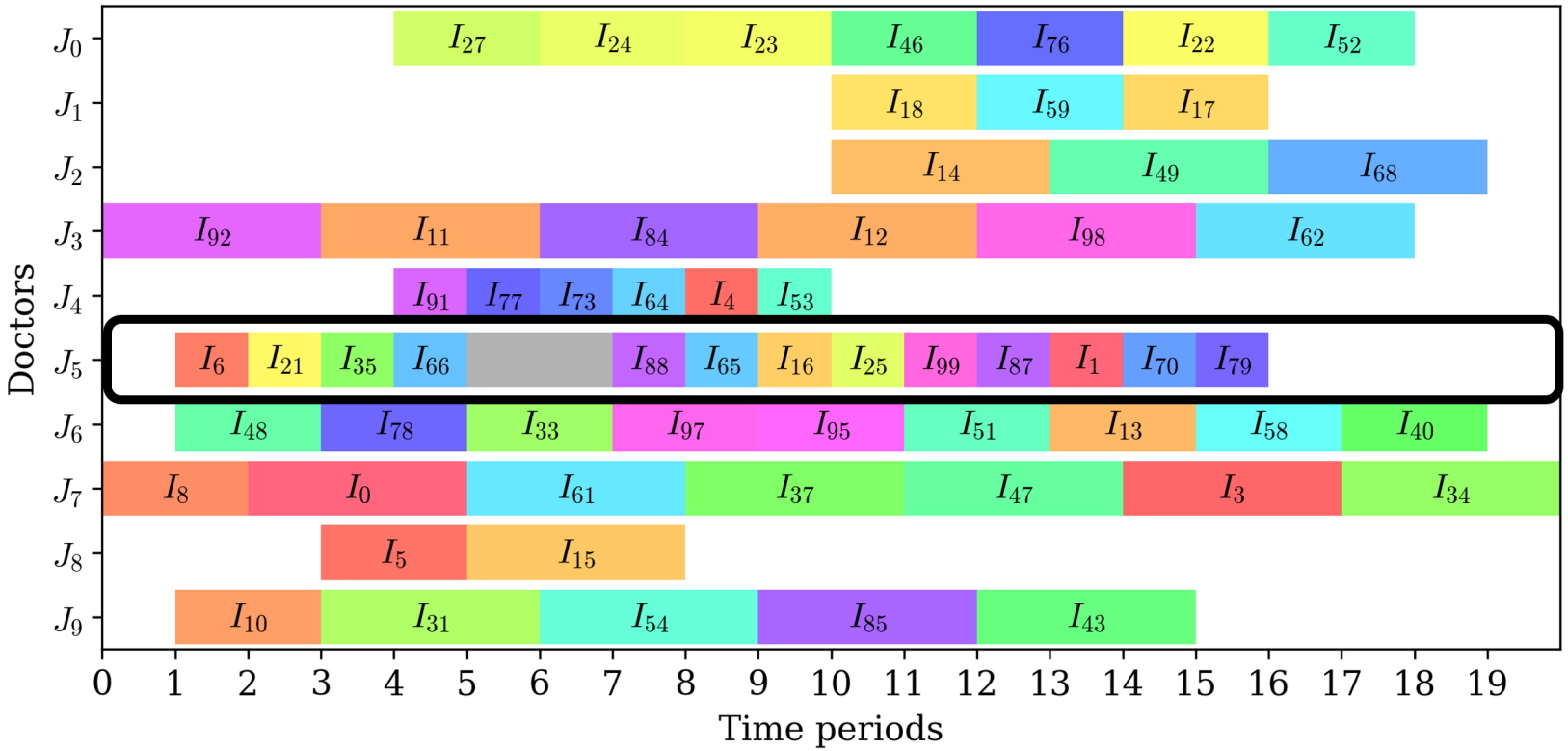


Figure: Example schedule for a doctor

Huge model

Doctor schedule formulation

Variables:

- Doctor schedules:

$$z_{j,s} \in \{0,1\} \qquad \forall j \in J,\ s \in S_j$$

where S_j is the subset of feasible schedules for doctor j

Constraints:

- Doctors assigned at most one schedule
- Patients assigned at most once

Huge model

Doctor schedule formulation

Constraints:

- Doctors assigned at most one schedule

$$\sum_{s \in S_j} z_{j,s} = 1 \qquad \forall j \in J$$

- Patients assigned at most once

$$\sum_{j \in J} \sum_{s \in S_j \mid i \in S_j} z_{j,s} \leq 1 \qquad \forall i \in I$$

Generating candidate schedules

Naive exhaustive search

Enumerate all feasible sequences of patients and start times for each doctor *a priori*, and compute objective contributions for each schedule

- Problem: combinatorial explosion even for small instances

Observation: many schedules are dominated by others

- For schedules for a doctor that include the same patients:
  - → The number of appointments and doctor satisfaction are identical
  - → Only patient satisfaction may differ
- Schedules with strictly lower patient satisfaction can be discarded

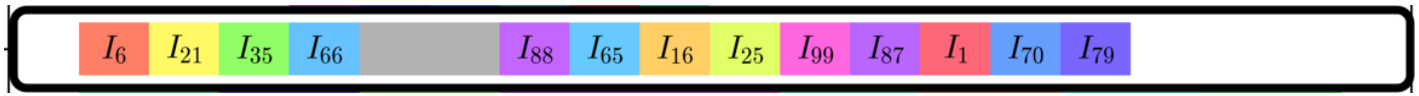


Figure: Every valid permutation of these appointments are generated, where one has the highest patient satisfaction

Optimised subset search

Key idea:

- Enumerate all feasible **subsets of patients** for each doctor
- For each subset, find the schedule that maximises patient satisfaction
- Only keep **non-dominated schedules**

Column generation algorithm

- Start with single-patient schedules
- Iteratively build larger sets by adding patients
- Solve small IP (using one of the compact formulations) for each subset to find best schedule
- Stop when no new feasible schedules can be generated

Column generation time

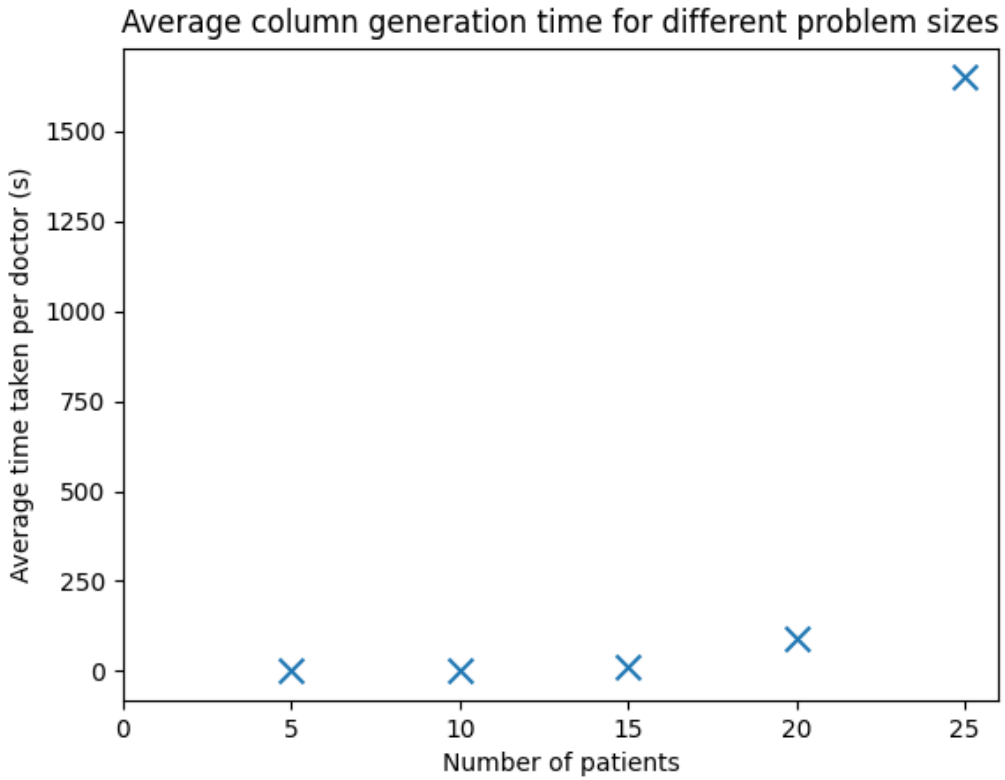


Figure: Average generation time vs number of patients (5 doctors, 2 diseases, 20 time periods)

Naive ran for 5 patients and generates 122176 schedules. The subset method generates for 25 patients, 145553 schedules.

Multi-processing

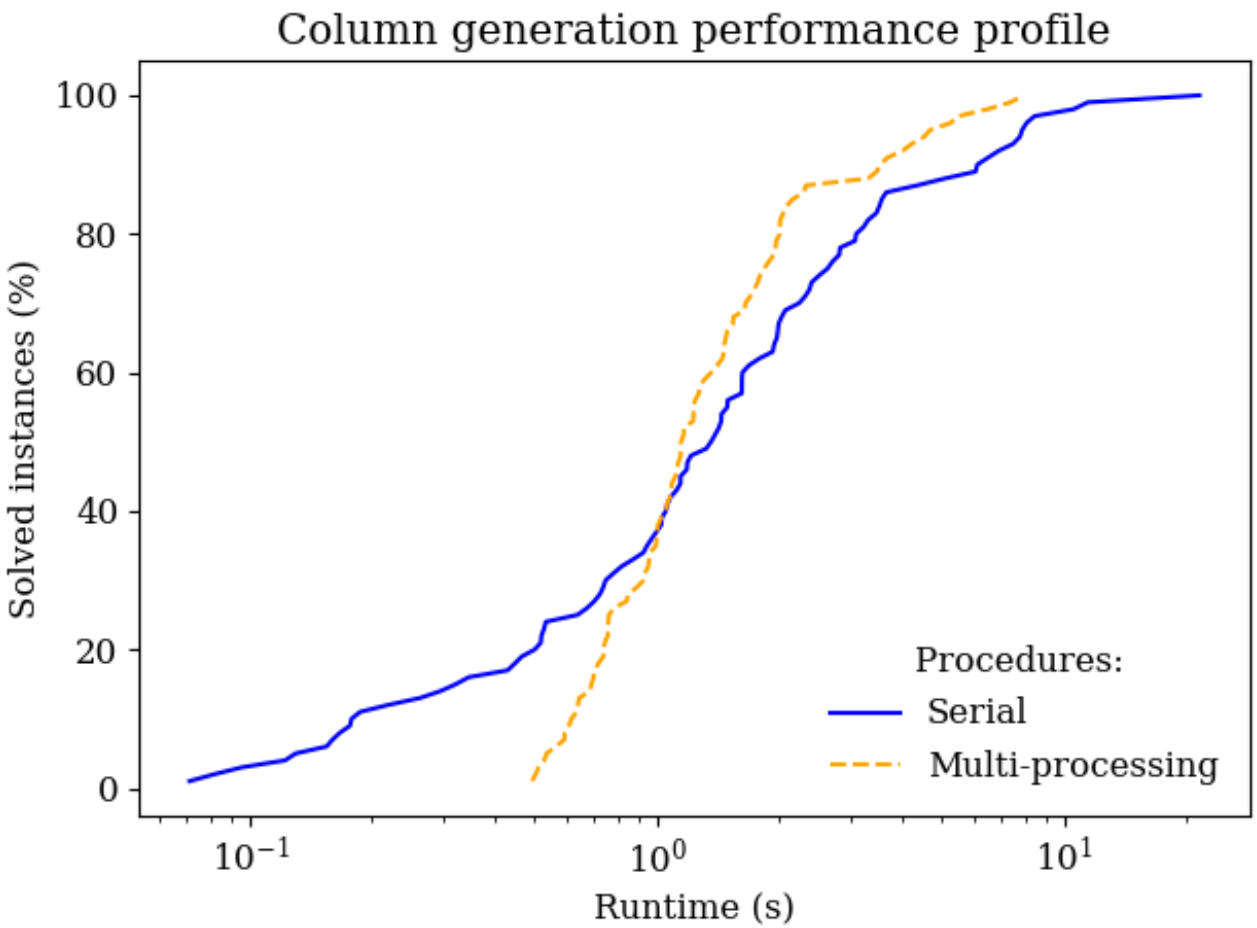


Figure: Performance profiles for column generation procedures
(100 instances with 20 patients, 5 doctors, 4 diseases and 10 time periods)

Generating schedule fragments

We explored the idea of generating **schedule fragments** as consecutive appointment blocks, to reduce the size of the columns generated

Key features:

- Each fragment represents a feasible mini-schedule of consecutive appointments for a single doctor
- Start times are explicitly specified to ensure appointments align without gaps or overlaps
- Fragments can later be linked to form full schedules that respect availability and treatment lengths
- Must generate fragments for each doctor separately because doctors take different amounts of time to treat diseases

Fragments model

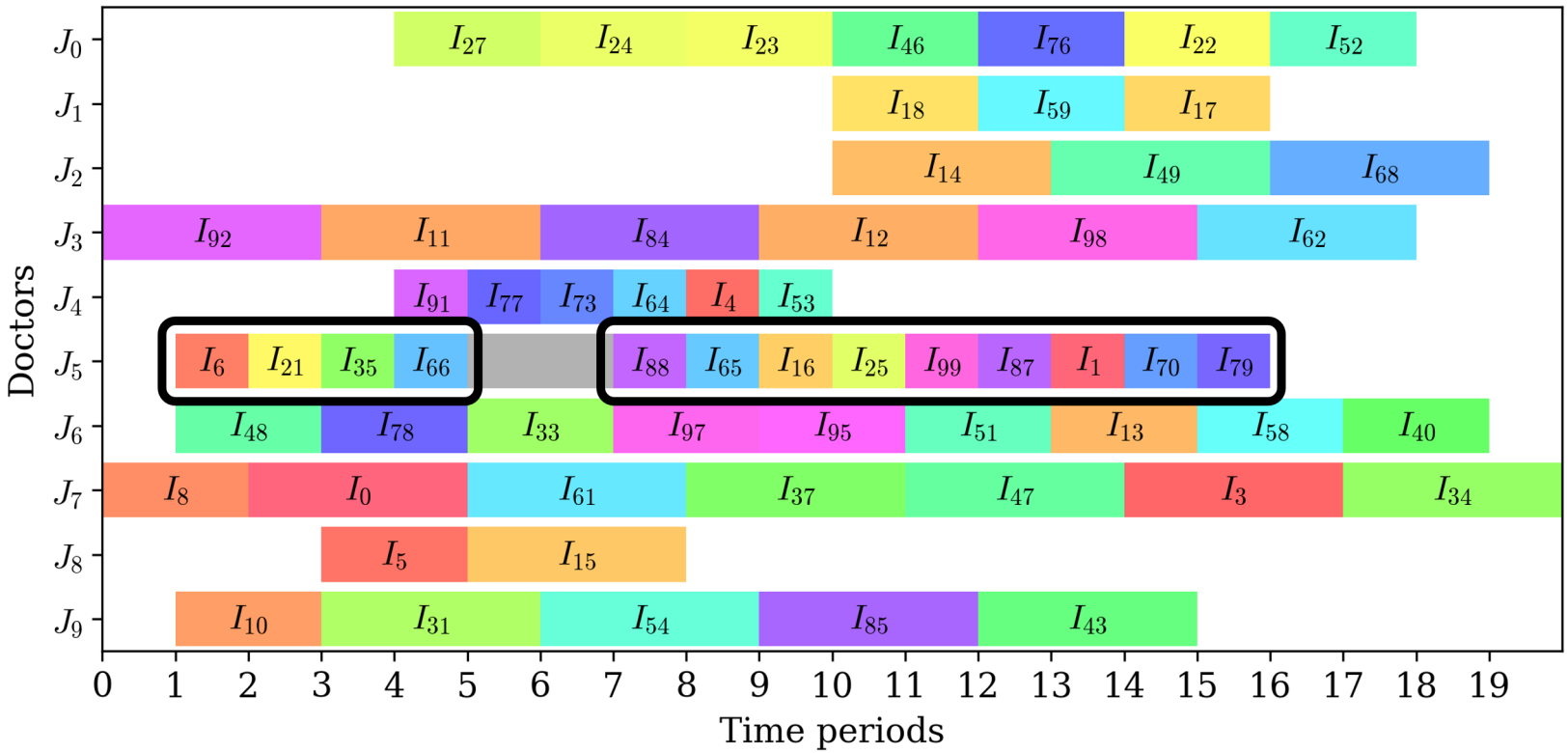


Figure: Example fragments of a doctor's schedule

Fragments model

Doctor schedule fragments formulation

Variables:

- Doctor schedules:

$$w_{j,f} \in \{0,1\} \qquad \forall j \in J,\ f \in F_j$$

where F_j is the subset of feasible fragments for doctor j

Constraints:

- Patients assigned at most once
- No overlap (over fragments)

Multi-objective optimisation

We aim to maximise three objectives simultaneously:

$$\begin{aligned} \max \quad & f_0(x) && \text{(Patient satisfaction)} \\ \max \quad & f_1(x) && \text{(Total appointments)} \\ \max \quad & f_2(x) && \text{(Doctor satisfaction)} \end{aligned}$$

subject to feasibility constraints $x \in \mathcal{X}$.

Epsilon-constraint method

- Choose one objective (f_0) as primary
- Convert the others into constraints:

$$f_1(x) \geq \varepsilon_1, \quad f_2(x) \geq \varepsilon_2$$

- Vary $(\varepsilon_1, \varepsilon_2)$ over feasible ranges to trace the trade-offs
- Pareto filtering then identifies non-dominated (efficient) solutions

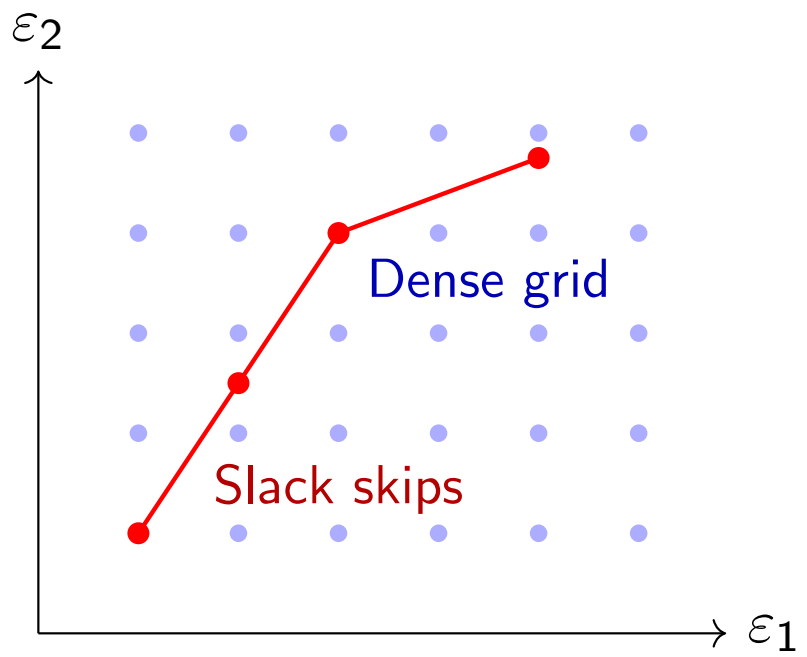
Pareto frontier

Dense grid search

- Iterates through all $(\varepsilon_1, \varepsilon_2)$ pairs, guarantees full Pareto coverage
- ⋆ Many redundant solves in dominated regions

Slack-based search

- Uses **slack values** $s_i = f_i(x^*) - \varepsilon_i$ to skip ahead by $\lfloor s_i / \Delta_i \rfloor$ steps
- Reduces computational effort by avoiding already-covered areas
- Idea: reuse "information" from prior solves to accelerate the search



Sketching the Pareto frontiers

Overview:

- Each point represents a feasible solution obtained from the multi-objective model
- The frontier traces the trade-offs between objectives, showing where improving one comes at the cost of another

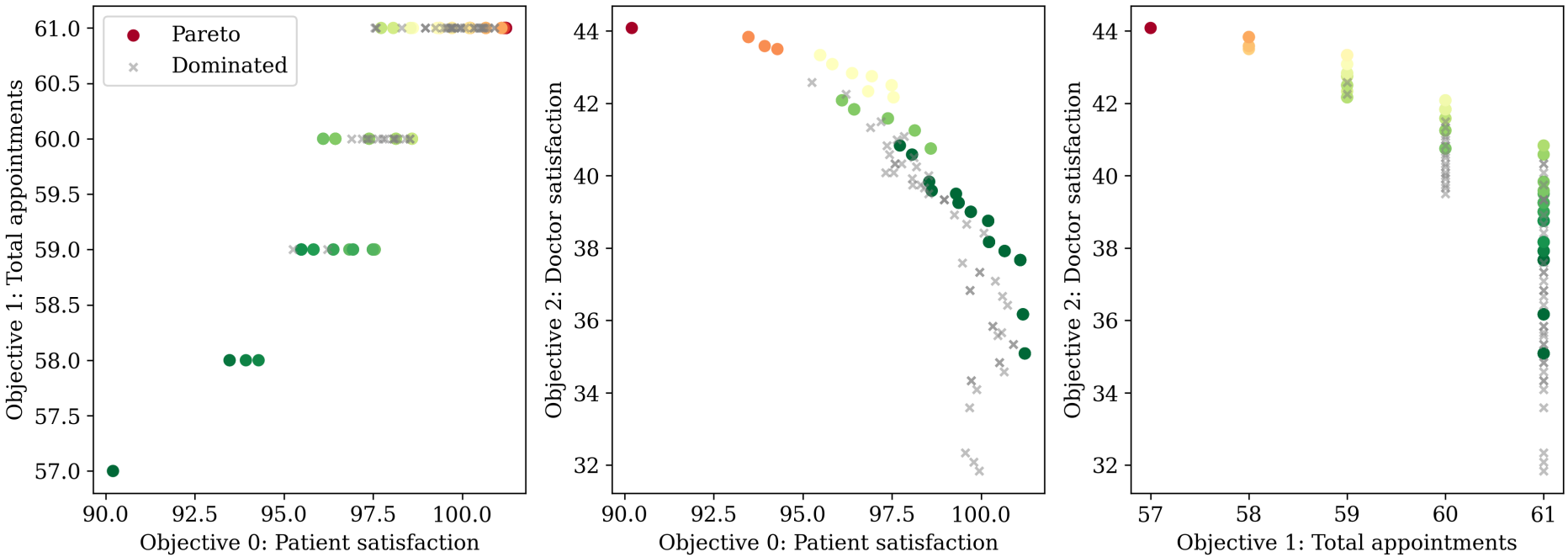


Figure: Example Pareto frontier diagrams for objective pairs

Implementation notes

Results and challenges

- The dense grid method works reliably and generates a complete Pareto frontier
- The slack-based approach has been implemented but currently shows inconsistent skipping behaviour

Next steps

- Debug slack update rules to ensure consistent skipping, by checking whether a potential $(\varepsilon_1, \varepsilon_2)$ pair is *un-dominated* before optimising
- Validate correctness by comparing recovered frontier with dense search results

Ideas

- **Partial column generation:** apply different solution methods for different doctor types within the one IP
  - → For lowly available and slow doctors, use the huge formulation (since the schedules can be generated very fast)
  - → For highly available and fast doctors, use a compact IP formulation or fragments to solve directly

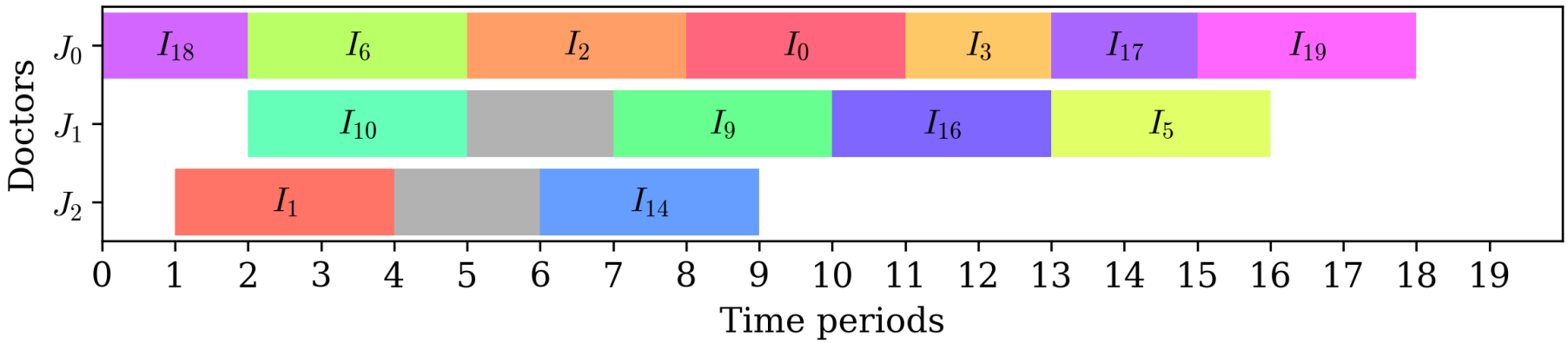
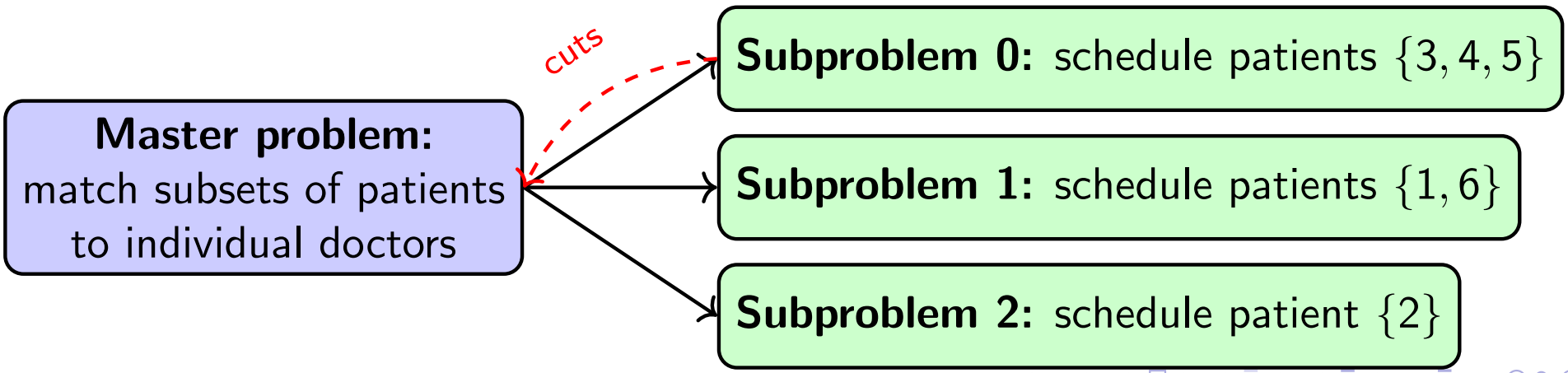


Figure: Lowly available vs. highly available doctors

Ideas

- **Benders' decomposition:** master problem matches patients to doctors, sub-problem schedules those patients who have been matched (similar to paper but simplified)
  - → Master problem ensures doctors are qualified and time windows are compatible
  - → Feasibility cuts ensure that subsets of patients can indeed be served a doctor (i.e. patients can be arranged within the doctor's availability)
  - → Optimality cuts ensure additional patients in a given subset increase the objective value for patient satisfaction; the other objective values are taken care of by the epsilon-constraint method



Thank you

Reference: Jiang et al. (2025). *Logic-based Benders decomposition for doctor-patient matching and scheduling considering chronic patients' online consultation time preference*. Computers & Operations Research 183: 107207. https://doi.org/10.1016/j.cor.2025.107207